

GUIA_ARQUITETURA.md

Arquivo

vou comer esse projeto me ajude ao inciar, criar o ambiente e codigo

Perfeito. Esse projeto está excelente para você aprender arquitetura real de API sem entrar em complexidade desnecessária.

Você vai construir:

- FastAPI
- arquitetura em camadas
- validação com Pydantic
- serviços
- persistência
- endpoints REST
- testes

E o melhor: exatamente no padrão usado em projetos profissionais.

GUIA_ARQUITETURA

1. Criando o ambiente do projeto

Criar pasta do projeto

Bash

```
cd focus-log-api
```

2. Abrir no VSCode

```
Bash
```

Se não funcionar:

```
Bash
```

ou abra manualmente.

3. Criar ambiente virtual

No Windows:

```
Bash
```

Ativar:

Bash

Se estiver no PowerShell e der erro:

PowerShell

Depois ativa novamente.

4. Criar requirements.txt

Crie o arquivo:

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
pydantic==2.7.1
pytest==8.2.0
```

Instalar:

Bash

5. Criar estrutura de pastas

No terminal:

```
mkdir app
mkdir app\routes
mkdir app\services
mkdir app\models
```

Bash

Criar arquivos `__init__.py` :

```
type nul > app\__init__.py
type nul > app\routes\__init__.py
type nul > app\services\__init__.py
type nul > app\models\__init__.py
```

Bash

6. Estrutura final

Você deve ficar com:

```
focus-log-api/
|
├─ app/
|   └─ main.py
|   └─ routes/
```

```
| | |└─ focus.py
| | |└─ diagnostico.py
| |
| |└─ services/
| | |└─ focus_service.py
| | |└─ diagnostico_service.py
| |
| |└─ models/
| | |└─ registro.py
| |
| |└─ database/
| | |└─ storage.py
|
└─ tests/
└─ requirements.txt
└─ README.md
```

7. Começando pelo MODEL (correto profissionalmente)

Crie:

app/models/registro.py

```
from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime
from enum import Enum
```

```
class CategoriaEnum(str, Enum):
```

```
coding = "coding"  
reuniao = "reuniao"  
estudo = "estudo"  
outro = "outro"
```

```
class RegistroFocoInput(BaseModel):  
    nivel_foco: int = Field(..., ge=1, le=5)  
    tempo_minutos: int = Field(..., gt=0)  
    comentario: str = Field(..., min_length=1)  
  
    categoria: Optional[CategoriaEnum] = None  
    tags: Optional[list[str]] = []
```

```
class RegistroFocoOutput(RegistroFocoInput):  
    id: int  
    criado_em: datetime
```

```
class DiagnosticoOutput(BaseModel):  
    total_registros: int  
    media_foco: float  
    tempo_total_minutos: int
```

Python

Executar

8. Criando o STORAGE

Agora você cria o “banco em memória”.

app/database/storage.py

```
from typing import List
```

Python

Executar

Extremamente simples no início.

Depois você pode migrar para SQLite.

9. Criando SERVICE de foco

app/services/focus_service.py

```
from datetime import datetime
from app.database.storage import registros

def criar_registro(dados):
    novo_registro = {
        "id": len(registros) + 1,
        "nivel_foco": dados.nivel_foco,
        "tempo_minutos": dados.tempo_minutos,
        "comentario": dados.comentario,
        "categoria": dados.categoria,
        "tags": dados.tags,
        "criado_em": datetime.now()
    }

    registros.append(novo_registro)
```

10. Criando SERVICE de diagnóstico

app/services/diagnostico_service.py

```
from app.database.storage import registros

def gerar_feedback(media):
    if media < 2:
        return "Muitas distrações hoje."
    elif media < 3:
        return "Foco abaixo do ideal."
    elif media < 4:
        return "Bom desempenho!"
    elif media < 5:
        return "Ótimo foco!"
    else:
        return "Estado de flow! 🚀"

def gerar_diagnostico():
    if not registros:
        return {
            "total_registros": 0,
            "media_foco": 0,
            "tempo_total_minutos": 0,
            "mensagem_feedback": "Nenhum registro encontrado.",
```



```
        "distribuicao_foco": {}
    }

    total = len(registros)

    soma_foco = sum(r["nivel_foco"] for r in registros)

    tempo_total = sum(r["tempo_minutos"] for r in registros)

    media = soma_foco / total

    distribuicao = {}

    for i in range(1, 6):
        distribuicao[str(i)] = len(
            [r for r in registros if r["nivel_foco"] == i]
        )

    return {
        "total_registros": total,
        "media_foco": round(media, 2),
        "tempo_total_minutos": tempo_total,
        "mensagem_feedback": gerar_feedback(media),
    }
```

Python

Executar

11. Criando as ROTAS

app/routes/focus.py

```
from fastapi import APIRouter
from app.models.registro import RegistroFocoInput
from app.services.focus_service import criar_registro
```

```
router = APIRouter()
```

```
@router.post("/registro-foco")
```

Python

Executar

app/routes/diagnostico.py

```
from fastapi import APIRouter
from app.services.diagnostico_service import gerar_diagnostico
```

```
router = APIRouter()
```

```
@router.get("/diagnostico-produtividade")
```

Python

Executar

12. Criando o MAIN

app/main.py

```
from fastapi import FastAPI

from app.routes.focus import router as focus_router
from app.routes.diagnostico import router as diagnostico_router

app = FastAPI(
    title="Focus Log API"
)
```

Python

Executar

13. Rodando o projeto

No terminal:

Bash

Vai aparecer algo assim:

```
Uvicorn running on http://127.0.0.1:8000
```

14. Abrindo Swagger

Acesse:

```
http://127.0.0.1:8000/docs
```

Aqui começa a mágica do FastAPI.

Você consegue:

- testar endpoints
- enviar JSON
- validar
- ver documentação automática

15. Primeiro teste POST

```
{  
  "nivel_foco": 5,  
  "tempo_minutos": 90,  
  "comentario": "Estudando FastAPI",  
  "categoria": "estudo",  
}
```

JSON

16. Próximo passo profissional

Depois disso vamos evoluir para:

- SQLite
- SQLAlchemy
- Repository Pattern
- autenticação JWT
- Docker
- deploy
- testes automatizados
- CI/CD

Mas primeiro:

1. faça funcionar
2. entenda cada camada
3. debug linha por linha

Esse é o caminho correto para virar backend de verdade.

       Fontes

agora vamos adicionar o sqlite

Excelente decisão.

Adicionar SQLite agora vai te ensinar:

- persistência real
- SQL
- conexão com banco

- separação correta da camada database
- preparação para ORMs futuramente

E o melhor:

o `sqlite3` já vem no Python.

1. O que vamos mudar

Antes:

```
lista em memória
```

Agora:

```
SQLite -> focus.db
```

2. Nova estrutura

A estrutura continua praticamente igual:

```
app/  
|  
├─ database/  
|   ├─ __init__.py  
|   └─ storage.py
```

Só que agora o `storage.py` será responsável pelo banco.

3. Criar o banco automaticamente

app/database/storage.py

Apague tudo e coloque:

```
import sqlite3

DATABASE_NAME = "focus.db"

def get_connection():
    conn = sqlite3.connect(DATABASE_NAME)

    conn.row_factory = sqlite3.Row

    return conn

def criar_tabela():
    conn = get_connection()

    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS registros (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nivel_foco INTEGER NOT NULL,
            tempo_minutos INTEGER NOT NULL,
            comentario TEXT NOT NULL,
```

```
        categoria TEXT,  
        tags TEXT,  
        criado_em TEXT NOT NULL  
    )  
""")  
  
conn.commit()
```

Python

Executar

4. Inicializar tabela no start da API

app/main.py

Atualize:

```
from fastapi import FastAPI  
  
from app.routes.focus import router as focus_router  
from app.routes.diagnostico import router as diagnostico_router  
  
from app.database.storage import criar_tabela  
  
app = FastAPI(  
    title="Focus Log API"  
)  
  
@app.on_event("startup")  
def startup():
```



```
criar_tabela()
```

Python

Executar

5. Atualizar o service de foco

Agora vamos salvar no SQLite.

app/services/focus_service.py

```
from datetime import datetime

from app.database.storage import get_connection


def criar_registro(dados):

    conn = get_connection()

    cursor = conn.cursor()

    criado_em = datetime.now().isoformat()

    tags = ",".join(dados.tags) if dados.tags else ""

    cursor.execute("""
        INSERT INTO registros (
            nivel_foco,
```

```

        tempo_minutos,
        comentario,
        categoria,
        tags,
        criado_em
    )
    VALUES (?, ?, ?, ?, ?, ?)
"""
    dados.nivel_foco,
    dados.tempo_minutos,
    dados.comentario,
    dados.categoria,
    tags,
    criado_em
))

conn.commit()

registro_id = cursor.lastrowid

conn.close()

return {
    "id": registro_id,
    "nivel_foco": dados.nivel_foco,
    "tempo_minutos": dados.tempo_minutos,
    "comentario": dados.comentario,
    "categoria": dados.categoria,
    "tags": dados.tags
}

```

Python

Executar

6. Atualizar diagnóstico

Agora vamos ler do banco.

app/services/diagnostico_service.py

```
from app.database.storage import get_connection
```

```
def gerar_feedback(media):  
    if media < 2:  
        return "Muitas distrações hoje."  
    elif media < 3:  
        return "Foco abaixo do ideal."  
    elif media < 4:  
        return "Bom desempenho!"  
    elif media < 5:  
        return "Ótimo foco!"  
    else:  
        return "Estado de flow! 🚀"
```

```
def gerar_diagnostico():  
  
    conn = get_connection()  
  
    cursor = conn.cursor()  
  
    cursor.execute("SELECT * FROM registros")  
  
    registros = cursor.fetchall()  
  
    conn.close()
```

```
if not registros:
    return {
        "total_registros": 0,
        "media_foco": 0,
        "tempo_total_minutos": 0,
        "mensagem_feedback": "Nenhum registro encontrado.",
        "distribuicao_foco": {}
    }

total = len(registros)

soma_foco = sum(r["nivel_foco"] for r in registros)

tempo_total = sum(r["tempo_minutos"] for r in registros)

media = soma_foco / total

distribuicao = {}

for i in range(1, 6):
    distribuicao[str(i)] = len(
        [r for r in registros if r["nivel_foco"] == i]
    )

return {
    "total_registros": total,
    "media_foco": round(media, 2),
    "tempo_total_minutos": tempo_total,
    "mensagem_feedback": gerar_feedback(media),
```

Python

Executar

7. Rodar novamente

Bash

8. O que vai acontecer

Agora vamos rodar o comando:

ChatGPT ▾

 Promoção gratuita ×

 Compartilhar ...

focus.dd

Esse arquivo é seu banco SQLite.

9. Visualizar banco graficamente

Recomendo MUITO instalar:

SQLite Viewer (VSCode)

ou

DB Browser for SQLite

DB Browser for SQLite

Você vai conseguir:

- abrir tabelas

- ver registros
- executar SQL
- aprender banco visualmente

10. Testando

POST:

JSON

```
{
  "nivel_foco": 4,
  "tempo_minutos": 120,
  "comentario": "Criando API com SQLite",
  "categoria": "coding",
  "tags": ["python", "fastapi"]
}
```

Depois:

GET /diagnostico-produtividade

11. Agora você já tem

Você acabou de transformar:

CRUD simples

em:

API backend real

com:

- FastAPI
- arquitetura em camadas
- SQLite
- persistência real
- services
- models
- validação

Isso já é MUITO mais próximo do mercado.

O próximo upgrade ideal será:

1. Repository Pattern
2. SQLAlchemy
3. Alembic
4. JWT Auth
5. Docker
6. Deploy

Aí começa nível backend profissional mesmo.



